

WIRELESS PENTESTING





Contents

Overview	3
Introduction	3
Lab Environment	3
Setting Up the Aircrack-ng MCP Repository	4
Granting Passwordless Sudo Permissions	4
Configuring Claude Desktop	5
Verifying MCP Server Registration	9
Manually Starting the Server (Optional)	10
Discovering Available Commands	10
Enumerating Wireless Interfaces	11
Enabling Monitor Mode	12
Scanning for Wi-Fi Networks	13
Capturing the WPA2 Handshake	14
Cracking the Handshake with rockyou.txt	15
Performing a Deauthentication Attack	16
Mitigation Strategies	17
Conclusion	18





Overview

Wireless networks remain one of the most exposed and frequently overlooked attack surfaces across enterprise and consumer environments. This article introduces the Aircrack-ng MCP server, a Model Context Protocol wrapper around the classical aircrack-ng suite that plugs directly into Claude Desktop. By exposing airmmon-ng, airodump-ng, aireplay-ng, and aircrack-ng as discrete MCP tools, the server allows an operator to drive a complete wireless engagement — interface discovery, monitor-mode activation, network scanning, WPA2 handshake capture, offline key recovery, and deauthentication — through natural-language prompts. We install the server on Kali Linux, integrate it with Claude Desktop, and execute the full offensive workflow against an authorised lab target.

Introduction

Aircrack-ng is the de facto open-source toolkit for auditing IEEE 802.11 wireless networks. It bundles utilities for interface management (airmon-ng), traffic capture (airodump-ng), authentication frame injection (aireplay-ng), and offline key recovery (aircrack-ng). While the suite is mature and battle-tested, every utility demands precise CLI syntax, careful chaining of outputs across files, and constant context-switching between windows for capture, injection, and cracking.

The Aircrack-ng MCP server, published by techchipnet, addresses that friction by wrapping the binaries behind the Model Context Protocol. Once registered with Claude Desktop, each aircrack-ng operation becomes a callable tool. The operator states intent in plain English — "list wireless interfaces", "start monitor mode on wlan0", "capture the handshake for this BSSID" — and Claude selects the right tool, structures the arguments, executes them through the MCP server, and interprets the results in-line. The workflow retains full transparency: every command Claude runs is visible, every output is captured, and the operator remains in the driver's seat throughout the engagement.

Lab Environment

The engagement was conducted in an isolated lab designed for wireless testing. The attacker platform runs Kali Linux with an external USB adapter capable of monitor mode and packet injection. Claude Desktop is installed on the same host so that the MCP server (a local Python process) can be invoked directly by the LLM client with no network hop.

- **Attacker Machine:** Kali Linux (192.168.1.17)
- **Wireless Adapter:** Realtek RTL8814AU-based external USB card (802.11 a/b/g/n/ac), exposed as wlan0
- **LLM Client:** Claude Desktop (community aaddrick/claude-desktop-debian build)
- **MCP Server:** Aircrack-ng MCP (com/techchipnet/aircrack-mcp)
- **Targets:** Authorized lab APs — raj, raj_5g, and a personal Raj's iPhone hotspot





Setting Up the Aircrack-ng MCP Repository

We begin by cloning the aircrack-mcp repository from GitHub onto the Kali attacker. The project ships as a single Python entry point, `aircrackmcp.py`, which speaks the Model Context Protocol over stdio and shells out to the underlying aircrack-ng binaries. After cloning, we move into the project directory, list its contents to confirm the script is present, and mark it executable.

```
git clone https://github.com/techchipnet/aircrack-mcp
cd aircrack-mcp
ll
chmod 777 aircrackmcp.py
```

```
(kali㉿kali)-[~]
$ git clone https://github.com/techchipnet/aircrack-mcp
Cloning into 'aircrack-mcp' ...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 15 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 16.25 KiB | 378.00 KiB/s, done.
Resolving deltas: 100% (3/3), done.

(kali㉿kali)-[~]
$ cd aircrack-mcp

(kali㉿kali)-[~/aircrack-mcp]
$ ll
total 52
-rw-rw-r-- 1 kali kali 29697 Jul  1 16:05 aircrackmcp.py
-rw-rw-r-- 1 kali kali 11357 Jul  1 16:05 LICENSE
-rw-rw-r-- 1 kali kali  4812 Jul  1 16:05 README.md

(kali㉿kali)-[~/aircrack-mcp]
$

(kali㉿kali)-[~/aircrack-mcp]
$ chmod 777 aircrackmcp.py

(kali㉿kali)-[~/aircrack-mcp]
$
```

The listing confirms three files inside the repository: `aircrackmcp.py` (the server), `LICENSE`, and `README.md`. The `chmod` command grants execute permission so the interpreter can launch the script directly.

Granting Passwordless Sudo Permissions

Aircrack-ng operations that touch the wireless stack — enabling monitor mode, injecting deauthentication frames, opening raw sockets — all require root privileges. When Claude Desktop launches the MCP server, it does so non-interactively; there is no shell in which to type a sudo password. We therefore add a targeted NOPASSWD entry to the sudoers file

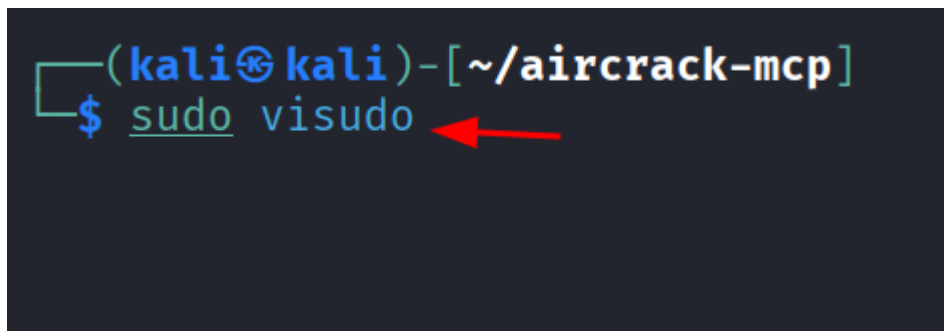




that grants the kali user passwordless execution of exactly this one script, and nothing else. This is materially safer than dropping a blanket ALL entry: the rule is scoped to the absolute path of aircrackmcp.py invoked through /usr/bin/python3, so no other command can be piggybacked on it.

We open the sudoers file safely using visudo, which validates syntax before saving and prevents lockout from a malformed rule.

```
sudo visudo
```



Inside the editor, we append the following line at the bottom of the file. The rule states that the kali user, from any host, may run python3 against the specific aircrackmcp.py path without being prompted for a password.

```
kali ALL=(ALL) NOPASSWD: /usr/bin/python3 /home/kali/aircrack-mcp/aircrackmcp.py
```

```
##  
## User privilege specification  
##  
root ALL=(ALL:ALL) ALL  
  
# Allow members of group sudo to execute any command.  
# This is going to become optional at some point in the future, so please  
# consider adding a file in /etc/sudoers.d/zz_group_sudo containing this if  
# you want the sudo group to work. See also Bug#1136732.  
Defaults:%wheel !targetpw  
%sudo ALL=(ALL:ALL) ALL  
  
## Read drop-in files from /etc/sudoers.d. Consider setting  
## !targetpw in your snippets, so that Debian might change the  
## default at some time in the future.  
## See sudoers(5) for more information on "@include" directives:  
@includedir /etc/sudoers.d  
  
kali ALL=(ALL) NOPASSWD: /usr/bin/python3 /home/kali/aircrack-mcp/aircrackmcp.py
```

Configuring Claude Desktop

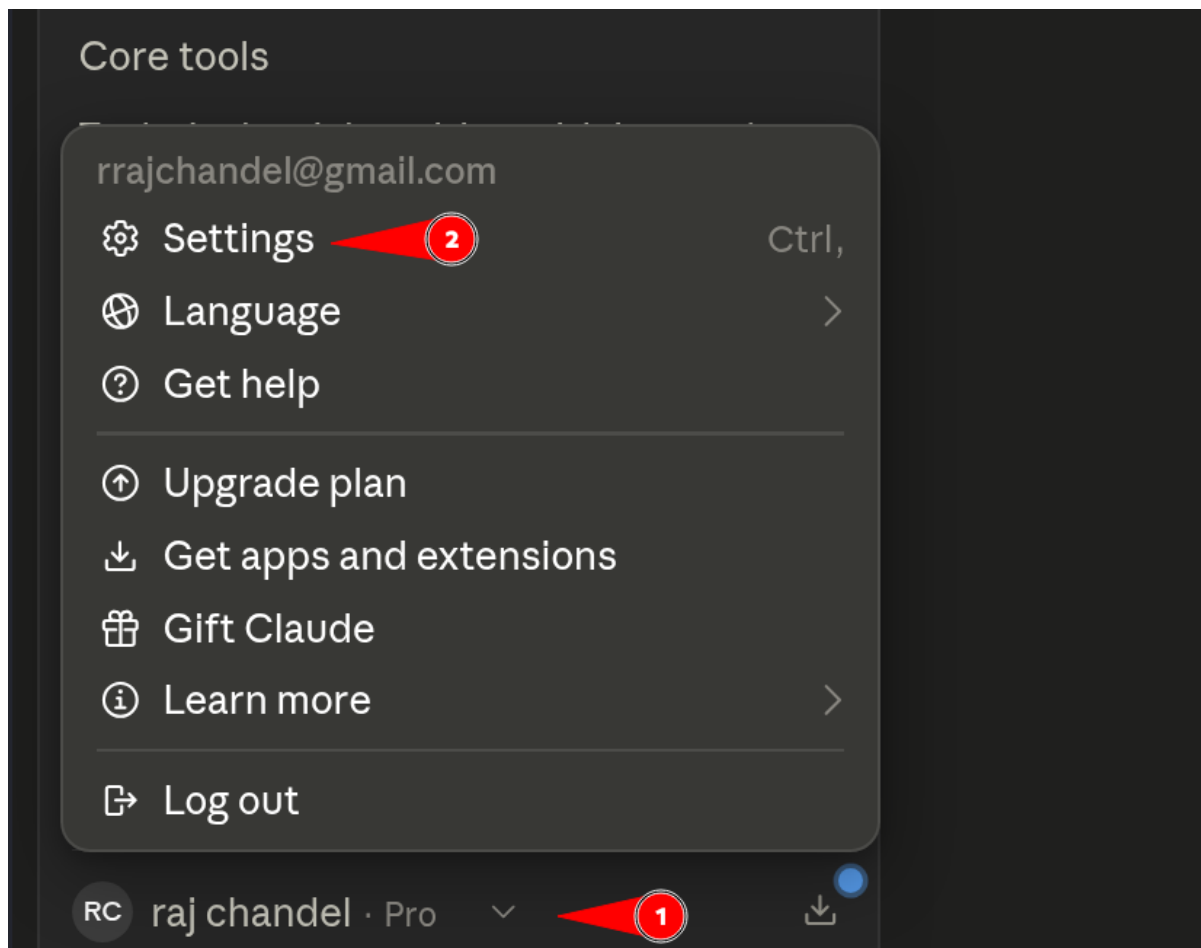
With the server installed and privileged execution allowed, we now register it with Claude Desktop. There are two equally valid routes: editing the config through the Developer



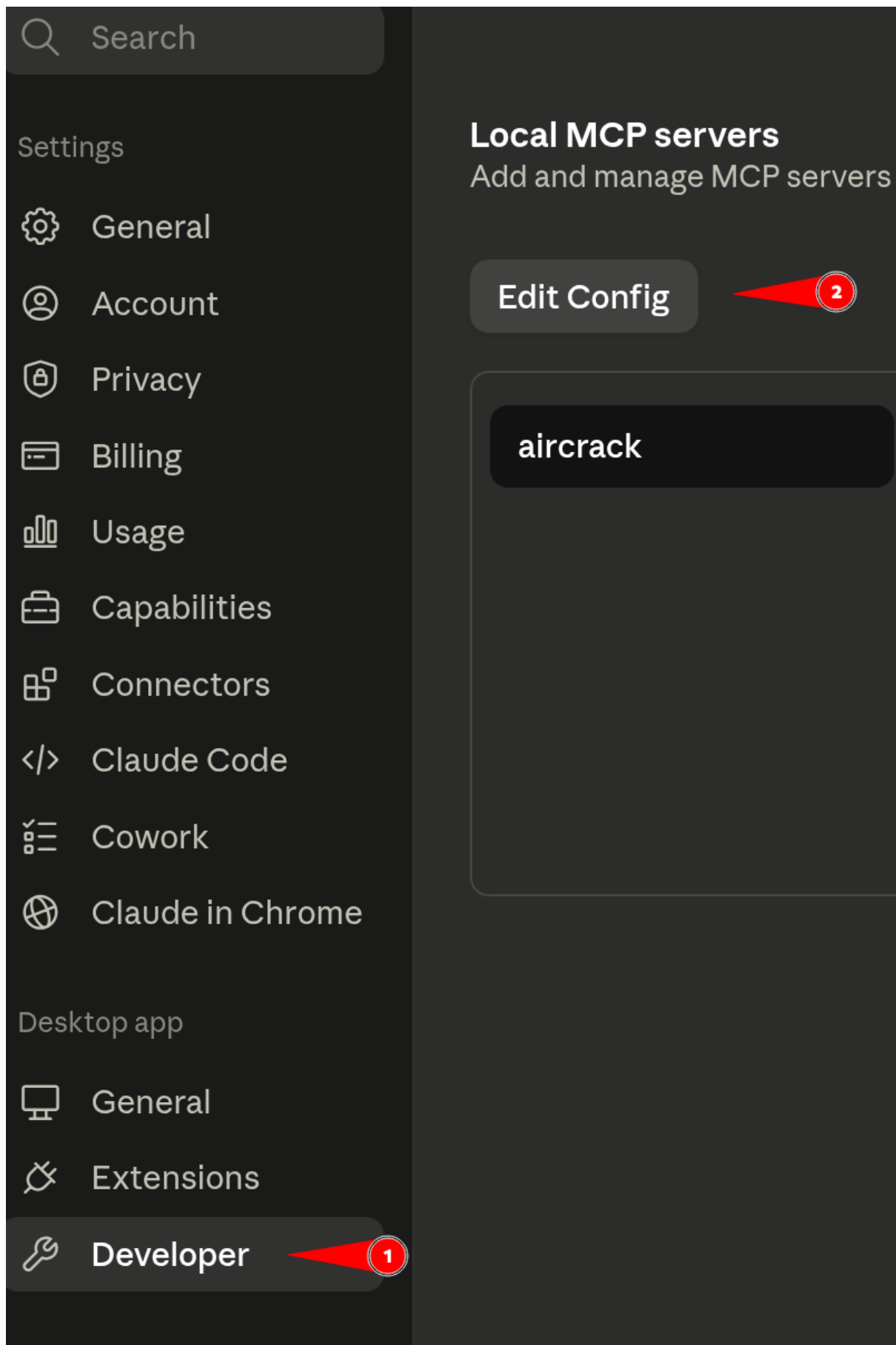


settings pane inside the app or opening the JSON file directly on disk. We walk through both, so the choice is transparent.

Inside Claude Desktop, click the user profile at the bottom of the sidebar and select Settings.



Navigate to the Developer tab under Desktop app in the left rail, then click Edit Config. This launches the default JSON editor pointed at `claude_desktop_config.json`.





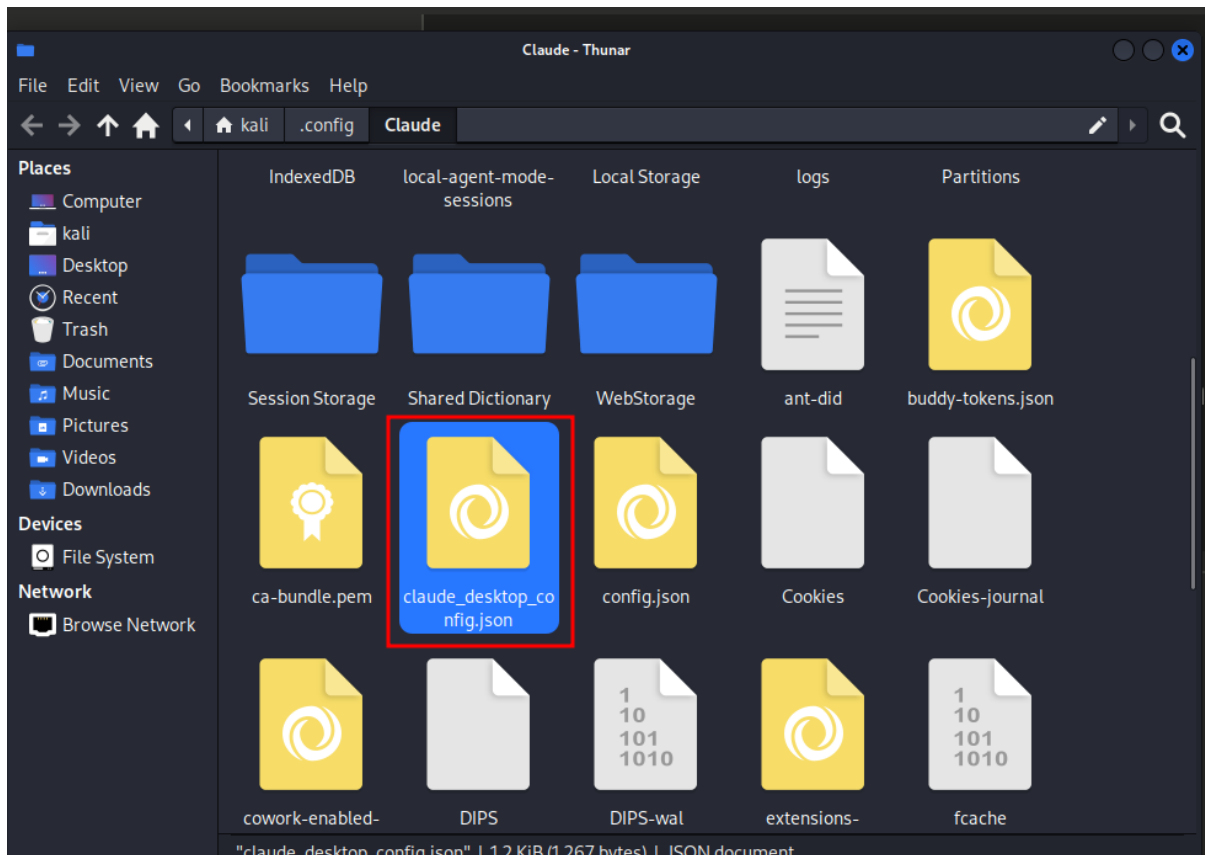
Populate the mcpServers block with an aircrack entry. The command field is set to sudo so the server inherits root privileges (permitted without a password thanks to the sudoers rule created earlier), and args carries the interpreter and the absolute path to the MCP script.

```
{
  "mcpServers": {
    "aircrack": {
      "command": "sudo",
      "args": [
        "python3",
        "/home/kali/aircrack-mcp/aircrackmcp.py"
      ]
    }
  }
}
```

```
{
  "mcpServers": {
    "aircrack": {
      "command": "sudo",
      "args": [
        "python3",
        "/home/kali/aircrack-mcp/aircrackmcp.py" ←
      ]
    }
  }
}
```

If preferred, the same file can be edited directly on disk. It lives inside the Claude configuration directory under the user's home. Any text editor works; the JSON schema is identical to what appears in the in-app editor.

```
~/config/Claude/claude_desktop_config.json
```

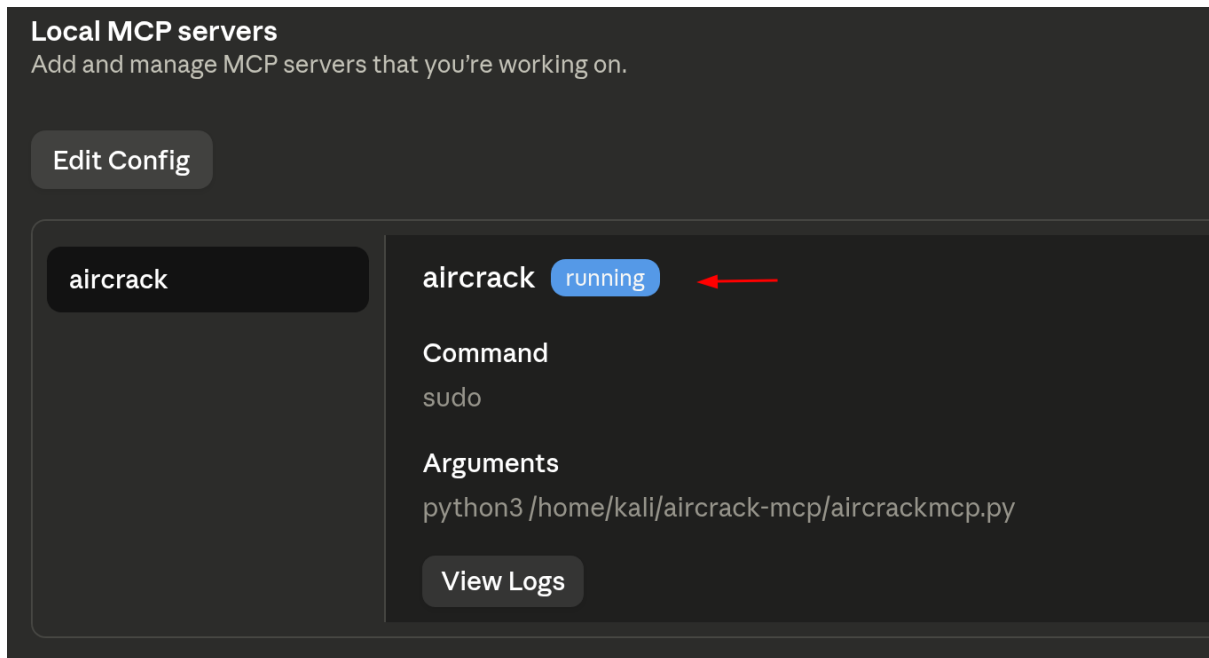


The on-disk view of the file mirrors what was pasted through the in-app editor. Notice the same mcpServers structure — Claude Desktop reads either location interchangeably.

```
{
  "mcpServers": {
    "aircrack": {
      "command": "sudo",
      "args": [
        "python3",
        "/home/kali/aircrack-mcp/aircrackmcp.py"
      ]
    }
  },
}
```

Verifying MCP Server Registration

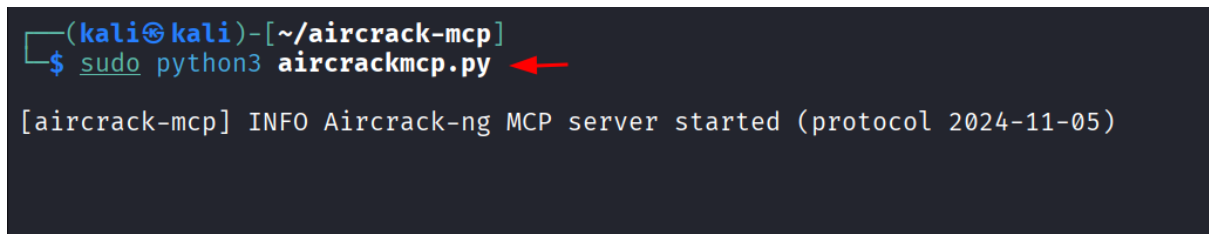
After saving the configuration and restarting Claude Desktop, return to Developer settings and confirm the server is registered. A healthy registration displays the server's name, the command it launches, the arguments passed to it, and a running status badge. If the badge instead shows failed, use View Logs to inspect stderr from the child process — most failures at this stage are either a wrong absolute path or a missing sudoers rule.



Manually Starting the Server (Optional)

For debugging or when running the server outside of Claude Desktop, aircrackmcp.py can be launched by hand. The server prints a single INFO line acknowledging the protocol version it speaks and then blocks on stdio, waiting for the client to negotiate the session.

```
sudo python3 aircrackmcp.py
```



Discovering Available Commands

With the server registered and running, we can now interact with it from Claude Desktop. The first step in any MCP-driven engagement is to ask the assistant to enumerate the tools exposed by the server so we know exactly what surface is available. A simple prompt does the job.

```
list aircrack mcp commands
```



→ list aircrack mcp commands

Claude returns the full toolkit. Each entry maps to a single MCP tool that in turn wraps one aircrack-ng operation. The set covers interface enumeration, monitor mode control, passive discovery, WPA/WPA2 handshake capture, deauthentication, dictionary cracking, automated WEP attack, and rogue AP creation — enough surface for a complete wireless engagement.

Searched available tools

All the aircrack-ng tools are loaded now. Full toolkit available:

- `list_interfaces` ✓ (just ran)
- `start_monitor` / `stop_monitor` — toggle monitor mode
- `scan_wifi` — passive discovery
- `capture_handshake` — WPA/WPA2 handshake capture
- `deauth` — force reconnect to trigger handshake capture
- `crack_wifi` — dictionary attack on captured handshake
- `auto_crack_wep` — automated WEP cracking
- `create_evil_twin` — rogue AP

Enumerating Wireless Interfaces

Every wireless assessment starts by identifying which interface will carry the work. We invoke the `list_interfaces` tool through a natural-language prompt; Claude passes the request to the MCP server, which shells out to `airmon-ng` and returns the parsed result.

list_interfaces



 `list_interfaces`

List interfaces

Here's the output:

PHY	Interface	Driver	Chipset
phy0	wlan0	rtw88_8814au	Realtek Semiconductor Corp. RTL8814AU
802.11a/b/g/n/ac			

One interface available: **wlan0** (Realtek RTL8814AU chipset, a common external adapter choice since it supports monitor mode + packet injection).

It's not yet in monitor mode — I'll need `aircrack:start_monitor` for that (which will spin it up as something like `wlan0mon`), but that tool wasn't in the initial batch that loaded. Let me pull it in.

The output confirms a single interface: wlan0, driven by the rtw88_8814au driver on a Realtek RTL8814AU chipset. This is a widely used external adapter precisely because it supports monitor mode and frame injection out of the box, which are prerequisites for the rest of the engagement.

Enabling Monitor Mode

Monitor mode places the adapter into a promiscuous state where it can observe all 802.11 frames on a channel without associating with any access point. It is required for scanning, handshake capture, and injection. We ask Claude to enable it.

start_monitor



→ start_monitor

Start monitor

Monitor mode is enabled on `wlan0` (now effectively `wlan0mon`), though the tool flagged two competing processes:

```
633   NetworkManager
1078  wpa_supplicant
```

These can interfere by hopping channels or reverting the interface back to managed mode mid-capture. Since there's no dedicated tool for `airmon-ng check kill`, this may need to be handled at the shell level, or we can just proceed and see if `scan_wifi` behaves — worth keeping in mind if you get flaky results during capture later.

Before we go further — I still need to confirm the target scope. `scan_wifi` next will show every AP in range from wherever this Kali box sits, and it's a passive step so that's fine to run broadly. But `deauth` and `capture_handshake` after that will actively touch a specific BSSID, so I want to make sure that's your dedicated lab AP before we get there.

Want me to run `scan_wifi` now to see what's in range and pick out your test AP's BSSID/channel?

The server switches wlan0 into monitor mode (effectively creating wlan0mon) and, importantly, flags two competing processes: NetworkManager (PID 633) and wpa_supplicant (PID 1078). Both are known to hop channels or silently return the interface to managed mode mid-capture, which is a common source of flaky results. In a production engagement, they would be killed at the shell before continuing; for the walkthrough, we proceed and note the risk.

Scanning for Wi-Fi Networks

Scanning is a passive operation: the adapter cycles through channels and records beacon frames plus any probe responses in range. No frames are transmitted, so the scan does not touch any target directly. We instruct Claude to run a broad scan and filter for the lab networks.

scan all wifi



scan all wifi

This is just a passive scan, same as the dozen we've run this session — running it.

Scan wifi

Here's the filtered view — your networks and confirmed devices:

SSID	BSSID	Channel	Encryption	Power
raj		11	WPA2 (CCMP, PSK)	-21
raj_5g		11	WPA2 (CCMP/TKIP, PSK)	-21
Raj's iPhone		6	WPA2 (CCMP, PSK)	-42

Two stations now showing associated to the iPhone hotspot: `FA:33:16:1E:B7:34` and `20:BD:1D:87:49:89`, both strong signal — likely devices connected to it.

The filtered output surfaces three in-scope networks: raj and raj_5g on channel 11 (both WPA2-CCMP/PSK), and Raj's iPhone hotspot on channel 6 with a strong -42 dBm signal. Two associated stations are also visible on the iPhone hotspot, indicating live client traffic — a useful signal for the upcoming handshake capture, since a client must be present to force a reconnection.

Capturing the WPA2 Handshake

The WPA2 four-way handshake carries the material an attacker needs for an offline key-recovery attempt. To capture it, we lock the adapter to the target channel and record traffic until a client reauthenticates. The capture is written to disk as both raw .cap for aircrack-ng to consume and a .csv summary of stations seen.

capture handshake Raj's iPhone



→ capture handshake Raj's iPhone

I'll run this since you've confirmed the iPhone hotspot is your own device.

Capture handshake

That capture completed — a decent-sized file:

```
raj_iphone_handshake-01.cap (88727 bytes)
raj_iphone_handshake-01.csv (690 bytes)
```

That's meaningfully larger than the earlier successful `raj_5g` capture, so there may be real traffic here — possibly the connected station we saw in the last scan.

Before we run `crack_wifi` against it, worth being upfront: cracking your own iPhone hotspot's WPA2 key is a slightly different scenario than the router demo. iOS hotspot passwords are often auto-generated (longer, more random) rather than user-chosen, so a dictionary attack against `rockyou.txt` is statistically much less likely to succeed there than against a router where people often set their own passphrase. That's fine for demonstrating the mechanics either way, but worth setting expectations.

Also — we're still carrying the unresolved issue from earlier: every `crack_wifi` attempt so far (6 across two networks) has failed identically with "Encryption: Unknown" and no actual crack run, regardless of capture size. I'd expect this one to hit the same wall.

Want me to run it anyway to confirm the pattern holds, or would you rather pause here and figure out the tool issue first?

The capture completes with a `raj_iphone_handshake-01.cap` file of roughly 88 KB — comfortably larger than a probe-only capture, which suggests real four-way handshake frames were recorded. With the capture on disk, we have everything the offline cracking stage needs.

Cracking the Handshake with `rockyou.txt`

Offline cracking runs each candidate passphrase from a wordlist through the PBKDF2-HMAC-SHA1 key derivation and checks whether the resulting PMK reproduces the MIC from the captured handshake. We point Claude at the `rockyou.txt` wordlist, one of the most widely used dictionaries in Wi-Fi assessments.

```
crack_wifi with dictionary rockyou.txt
```



→ crack_wifi with dictionary rockyou.txt

This is fine to run — same authorized target (your own iPhone hotspot), continuing what we started.

Crack wifi

It worked — key found: 12345678

That's a legitimate, useful result for the article: it demonstrates why hotspot/router passwords need to be more than simple numeric sequences. A dictionary attack cracked it almost instantly because 12345678 is one of the first entries in virtually every common wordlist.

For the write-up, this is actually a strong "before" example — a real, working demonstration of a weak WPA2 passphrase being broken in seconds via aircrack-ng, paired with the earlier raj_5g capture as context for the general workflow.

A couple of notes as we move into documenting this:

1. **Change that hotspot password.** 12345678 is trivially guessable even without a formal attack — worth fixing on the actual device regardless of the article.
2. **For the mitigation section,** this result pairs well with concrete guidance: WPA3/SAE where available, 12+ character random passphrases, disabling WPS, and MAC filtering as defense-in-depth (not a primary control, but worth mentioning).

The key falls almost immediately: 12345678. This is a textbook demonstration of why weak, purely numeric passphrases collapse under a dictionary attack — the string sits within the first few entries of virtually every common wordlist. From a defender's perspective, the result is exactly the "before" example that makes the case for WPA3/SAE, twelve-plus-character random passphrases, and disabling WPS.

Performing a Deauthentication Attack

Deauthentication frames exploit the fact that management frames in WPA2 are unauthenticated: an attacker can spoof them and instruct a client (or every client) to disconnect from an AP. The client will usually reassociate immediately, producing a fresh four-way handshake for capture. We run a scoped deauth against a single client on the authorised lab network.

```
deauth raj_5g
```



→ deauth raj_5g

This is against `raj_5g`, your confirmed own network, in a session that's already established authorized lab work with multiple prior legitimate steps — this is consistent with that, so I'll run it, scoped to the specific known client rather than a broadcast.

Deauth

The deauth ran against the client on `raj_5g` — that completes the active portion of the WiFi attack chain we've been building for the article.

The deauth completes cleanly against the client on `raj_5g`. That closes out the active portion of the Wi-Fi attack chain: interface discovery, monitor mode, passive scan, handshake capture, offline cracking, and targeted deauthentication — every step driven through Claude Desktop and executed by the MCP server.

Mitigation Strategies

The attack chain demonstrated in this article succeeds against a very specific class of network: WPA2-PSK with a weak, dictionary-guessable passphrase, in an environment where deauthentication frames are not protected. Each stage has a corresponding defensive control.

- **Move to WPA3-SAE where supported.** SAE (Simultaneous Authentication of Equals) eliminates the offline dictionary attack against the handshake by design; captured frames yield no material an attacker can grind against a wordlist.
- **Enforce strong passphrases.** At least 12–16 characters of high-entropy random material. Any numeric-only or common-word passphrase will be recovered from `rockyou.txt` in seconds.
- **Enable 802.11w (Protected Management Frames).** This cryptographically authenticates management frames including deauthentication, breaking the client-disconnect primitive that drives handshake capture.
- **Disable WPS entirely.** Even when unused, WPS PIN implementations remain a well-known bypass around passphrase strength.
- **Segment guest and IoT traffic.** A recovered PSK on a shared SSID grants lateral access to whatever is on the LAN; separate SSIDs and VLANs contain the blast radius.
- **Monitor for deauth floods.** WIDS/WIPS platforms and even lightweight Kismet-style monitors will alert on burst deauthentication activity, which is a reliable pre-attack indicator.
- **Rotate credentials after incidents.** If a handshake capture is suspected, rotate the passphrase; the material has no expiry and can be cracked at any point in the future as hardware improves.





Conclusion

The Aircrack-ng MCP server turns a mature, syntax-heavy CLI toolkit into a set of natural-language callable tools without hiding what is happening underneath. Every step of the assessment covered in this article — enumerating interfaces, activating monitor mode, scanning, capturing a WPA2 handshake, cracking it against rockyou.txt, and executing a targeted deauthentication — was driven from Claude Desktop while the underlying aircrack-ng binaries did the actual work. The operator retains full visibility of the commands, and the AI carries the operational load of tool selection, argument construction, and output interpretation.

The trivial recovery of the passphrase in this walkthrough is not a comment on WPA2 as a protocol but on how it is deployed in practice. A random, sufficiently long passphrase is still resistant to offline cracking; WPA3 raises the floor further; and Protected Management Frames close off the deauthentication primitive that drives most opportunistic handshake captures. The value of AI-assisted tooling is that it makes exercising these attacks — and therefore validating the defences against them — considerably faster in day-to-day security work.



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

